Autoscaling LLM Inference on Kubernetes: CPU-Driven Elasticity of a llama.cpp Service under HPA

Student Name 1
Matricola XXXXXXX
Sapienza University of Rome
name1.XXXXXXX@studenti.uniroma1.it

Student Name 2
Matricola XXXXXXX
Sapienza University of Rome
name2.XXXXXXX@studenti.uniroma1.it

Abstract*—We deploy a small-parameter LLM (Qwen3.5-0.8B) served by llama.cpp inside a Kubernetes pod behind a thin FastAPI proxy, and evaluate whether a CPU-based Horizontal Pod Autoscaler (HPA) is a sound elasticity mechanism for LLM inference, and at what cost. The service runs on two `t3.medium` workers with the HPA targeting 60% CPU between 1 and 2 pods. A first campaign establishes elasticity and capacity: under a continuous user ramp (Test A, N=5) the HPA scales out 1→2 and back 2→1 in every run, and under fixed user intensities from 10 to 50 (Test B, N=5 per level, 25 runs) the deployment saturates at two pods, with the p95 response time pinned against the proxy's 300 s timeout and error rates (503/504/502) rising to 34.7% overall. Attributing end-to-end delay per request shows that the llama decode dominates (40 s) while the orchestrator and transport path contribute only ≈11 ms, so the bottleneck is the model container, not Kubernetes. A second campaign varies the HPA ceiling: with `maxReplicas` 4 (exp4) and 6 (exp6), N=20 per level, availability improves from 0.73–0.96 to 0.76–1.0, p95 falls from 51–101 s to 30–98 s, and at level 50 exp6 processes 7 × the requests of exp4 by absorbing the queue instead of dropping it. Size-isolated runs (Test C) separate the per-class cost of a request and expose cross-size interference: a small request takes 25 s in isolation but 88 s when mixed with medium and large ones. A bursty workload (Test D) shows the HPA reacts to a sudden 6 × step in users within 27 s (CPU 0%→86–106%) with zero errors over 145 requests, although short bursts are absorbed by the service queue rather than rejected. Projected over six months, the autoscaled cluster costs $513, versus $749 for a single peak-sized `t3.xlarge` without elasticity and $3,787 for an equivalent AWS Lambda deployment.

I. Introduction

LLM inference is an unusual target for autoscaling. Each request is long-lived, compute-bound, and variable in cost: on CPU-only hardware a single generation occupies a core for tens of seconds while the model decodes one token at a time. Whether a reactive autoscaler can serve such a workload usefully is not obvious. HPA-like controllers react to a measured signal (here CPU), but the signal itself is the work being done, and the work takes longer than the reaction horizon. This report does not ask whether Kubernetes can add pods; it asks whether the CPU of an LLM service is a useful elasticity signal, what the deployment is capable of at its limits, and what it costs.

We report two campaigns, and the relationship between them is the spine of this paper. Campaign A (Sec. VI) measures the baseline deployment. Test A drives it with a continuous ramp of users and shows that the HPA deterministically scales out 1→2 and back 2→1 (scale-*in* is the behaviour almost no experiment of this kind shows). Test B sweeps fixed user counts from 10 to 50 and locates the saturation point: the two workers are the ceiling, and beyond roughly 20–30 users the p95 latency is pinned against the proxy's 300 s timeout while the error rate climbs. Delay attribution over per-request timestamps establishes where the time goes and isolates the bottleneck. Campaign B (Sec. VII) is the response to that ceiling: it raises the HPA limit to four and then six pods, and shows monotonic improvement in latency, errors and availability.

That second campaign produced the result we expected to see confirmed rather than discovered: the elasticity variable is the per-request service time, set almost entirely by the model, with the orchestrator contributing a constant ≈11 ms. Varying request size isolates this cleanly — a small generation costs 25 s, a large one 90 s, at the same pod count — and the mixed workload shows the queueing penalty a long tail imposes on short requests. A burst experiment shows the practical edge of the design: a sudden 6 × step in users is absorbed without a single error, because the queue drains during the following lull.

Section II gives the background, Sec. III positions the work, Sec. IV describes the application, Sec. V the methodology, Secs. VI and VII the two campaigns, Sec. VIII the cost evaluation, and Sec. IX the limitations.

II. Background

A. Kubernetes and the Horizontal Pod Autoscaler

Kubernetes schedules application instances (pods) on worker nodes. The Horizontal Pod Autoscaler (HPA) reads a resource signal — here average CPU utilisation across the pods of a deployment — and, when the utilisation crosses a configured target, changes the number of replicas between a configured minimum and maximum. Scaling is governed by two stabilisation windows: scale-*out* may start within seconds, while scale-*in* is delayed (300 s by default) so that transient dips do not collapse the pool. The signal comes from the Metrics Server, which reads CPU usage from `cgroup` accounting. Two properties of this mechanism matter for our workload. First, the signal is a *percentage of the CPU request*: a pod that requests 1700 m of the 2000 m a `t3.medium` offers will read near 100% whenever the model is decoding. Second, HPA acts on an average and reacts to the recent past, so the latency of the reaction is bounded by how long the CPU has already been saturated.

B. llama.cpp and CPU-based LLM inference

llama.cpp is a CPU-first inference engine for quantised transformer models. Generation is autoregressive: the model emits one token at a time, and each token consumes the full model compute. On two virtual cores, our 0.8B-parameter model (Q6_K quantisation, 791 MB) decodes at ≈26 tok/s (`MEASURE.md`), so a 256-token response takes roughly 10–20 s of sustained single-core work. With `-parallel 2` each instance holds two decode slots, so a pod can serve two generations concurrently, after which requests queue inside the proxy. This is the entire mechanism of our load: CPU is the work, and the work is a long decode.

C. The experimental environment

The deployment runs on AWS Academy Learner Lab, a sandbox with hard caps (≤8 instances, ≤31 vCPU, instance size ≤medium) enforced by the course. Sessions last about two hours, credentials are temporary and region-scoped (us-east-1 or us-west-2), and the lab auto-restarts instances left stopped, so every experiment is a fresh cluster built from scripts (`infra/`) and torn down afterwards. Load generation must stay inside the AWS perimeter (Learner Lab guidance); all client-side figures below are intra-region values and lower bounds on what an external user would see.

III. State of the Art

We position our results against the published literature on container autoscaling and on the economics of provisioned versus serverless compute, and state at the end which of our findings confirm prior work and which do not.

A. Autoscaling of containers

Casalicchio and Perciballi [1] show that the choice between relative and absolute scaling metrics materially changes autoscaler behaviour, and Casalicchio and Iannucci [2] survey container orchestration more broadly. Our experiment is a single, concrete instance of their general point: the metric that governs our autoscaler is CPU as a fraction of a request, and the shape of that signal is set by the model's decode rate. Pod autoscalers that react to request counts would behave differently for exactly the workload we study.

B. Autoscaling LLM inference

Most published LLM serving work assumes GPUs and measures token throughput and batch efficiency rather than elasticity under a load profile; a CPU-only deployment with a single small model is closer to the classic capacity-planning setting. The general lesson we take from the serving literature is that service time is a first-class workload parameter: per- request duration varies by an order of magnitude with the requested output length, which is precisely the variable that makes the product "arrival rate × service time" (Little's Law [3]) the correct capacity input rather than arrival rate alone.

C. Economics: provisioned versus serverless

Adzic and Chatley [4] report large savings moving to serverless, and Eivy and Weinman [5] show the same conclusion reverses once traffic is steady and predictable. Our R4 evaluation (Sec. VIII) finds the same trade-off on an LLM workload: a peak-sized single instance without elasticity costs more than the autoscaled cluster, and a serverless deployment is roughly 7× more expensive because long CPU-bound inferences are billed at serverless unit prices for every second of compute.

D. Position of this study

Three of our results restate known ones on a new workload: reactive autoscalers need headroom and react on a delayed signal; serverless loses to provisioned for long steady compute; and the bottleneck of a distributed service is usually not the orchestrator. Two are less standard. The clean 2→1 scale-*in* under an idle drain is rarely shown in this literature because most load profiles never return to idle; Test A demonstrates it in every replicate. And the ≈11 ms orchestrator share of a 40 s request is a direct measurement, using a timing header added to the proxy, of the claim that Kubernetes itself is cheap next to the work it schedules.

IV. Application

A. Architecture

A client sends an OpenAI-compatible `POST /generate` request to the FastAPI proxy inside a pod. The proxy streams the request to a llama-server sidecar on `localhost:8080`, which decodes the model and streams tokens back. A shared `emptyDir` volume carries the quantised GGUF model, placed there by an `initContainer` before the main containers start. Each pod therefore requests 1700 m CPU (llama) + 100 m CPU (proxy), which fits one `t3.medium` and lets two pods exactly fill the two workers. The HPA targets 60% average CPU with `minReplicas` 1 and `maxReplicas` 2. A NodePort service exposes port 30080. The load generator runs on a separate `t3.micro` inside the AWS perimeter.

B. Workload

Three request classes span the resource-usage space by capping the generated output length: `small` (`max_tokens` 32), `medium` (128) and `large` (256). Output length, not input size, is the variable we vary, because on this workload it is the quantity that multiplies decode time. The mix workload draws a size per request with probabilities 0.5/0.3/0.2 (small/medium/large).

TABLE I
MAPPING OF THE EVALUATION ONTO THE STANDARD PROCEDURE FOR MEASUREMENT-BASED CLOUD SERVICE PERFORMANCE EVALUATION

Step	Where addressed
1. Purpose, scope	Sec. V; client and server side
2. Features	LLM inference pod; auth, data store and front end excluded by assumption
3. Metrics	Sec. V-A
4. Tool	Locust 2.46 headless with custom `LoadTestShape` profiles (ramp, burst)
5. Design	Sec. V-C; ramp, capacity sweep, size isolation, burst; N=5 (Test A/B) and N=20/level (variants)
6. Environment	Sec. V-E
7. Execution	Sec. VI, Sec. VII

C. Instrumentation

Two measurements are measurement infrastructure rather than application logic. First, the proxy adds an `X-Upstream-Ms` header carrying the proxy→llama round-trip, so the per-request delay can be split as

$$\text{orchestrator+transport} = \text{total} - \text{upstream},$$

where *total* is the client-side response time and *upstream* is the header. Second, a collector on the master (`infra/collect.sh`) samples `kubectl top pods`, replicas, HPA state and events every minute (20 s in the burst test) into per-run CSV files, aligned by UTC timestamp with Locust's per-request detail log.

V. Performance Evaluation Methodology

Table I maps the evaluation onto the standard procedure for measurement-based performance evaluation of cloud services.

A. Metrics

User-oriented metrics come from the generator: response time (median, p95), successful requests, achieved throughput, and error rate. *System-oriented* metrics come from the collector: pod count, HPA state, CPU utilisation and autoscaling events. Availability is reported in two forms: end-to-end success rate (2xx over all attempts) and, where relevant, reachability of the service. Throughput is successful requests per second.

B. Tool selection

We chose Locust 2.46: Python (already the project language), headless with CSV output (`locust_stats.csv`, `locust_failures.csv`), a `LoadTestShape` API for custom profiles, and per-request hooks that write the timing detail used in the delay split. The generators are *closed-loop*: a fixed number of simulated users issues requests back-to- back, so achieved rate is a consequence of concurrency and service time rather than an input. We accept this because the quantity we control is the number of concurrent users, which is how a real audience behaves.

C. Load profiles

Four profiles cover the questions. **Test A** (elasticity) ramps users continuously to a maximum of 12, holds, then drains to zero with an idle period long enough for scale-*in*; `ramp_shape.py`. **Test B** (capacity) holds a fixed user count (10/20/30/40/50) with a 6-minute steady phase, N=5 runs per level; `exp-b.sh`. **Test C** (size isolation) fixes the workload size per run (small/medium/large/mix) at 20 users with a 3-minute steady phase, 10 runs per size, on six fixed pods (12 parallel decode slots); `day-run.sh`. A 90 s drain between runs and a pod restart every five runs keep the server out of the degradation region. **Test D** (burst) alternates a 2-user baseline with 12-user bursts, 120 s normal / 60 s burst, two cycles per run, N=3; `burst_shape.py`.

D. Service level objectives

We fix the following objectives before the results, at the level a small internal text-derivative service could commit to:

- **Availability** $\geq$ 99% end-to-end success at design load ($\leq$20 concurrent users).
- **Error rate** below 10% at design load.
- **Interactive latency:** p95 below the 300 s proxy timeout for the small class, the class a user waits on synchronously.
- **Elasticity:** scale-out 1→2 within 60 s of the CPU target being crossed, and scale-in 2→1 after an idle drain.

Section VII-D reports compliance. We state these before the results because an objective chosen after seeing the data is not an objective.

E. Experimental environment

All runs used 1 master `t3.small` and 2 workers `t3.medium` (Amazon Linux 2023, Kubernetes v1.36, Flannel, Metrics Server). Variant campaigns used a 6-worker cluster (exp6) with the same control plane. The model is `unsloth/Qwen3.5-0.8B-MTP-GGUF:UD-Q6_K_XL`, served by `ghcr.io/ggml-org/llama.cpp:server` build 10380 with `-threads 2 -parallel 2 -reasoning off`. The load generator ran on a `t3.micro` in the same region. Every experiment ended with a full teardown (`just cluster-down`).

VI. Campaign A: Elasticity and Capacity

A. Elasticity under a continuous ramp (Test A)

Test A drives the deployment with a continuous user ramp (warm-up, ramp to 12 users, steady, ramp-down, idle drain). Five replicates (N=5), all complete (`interrupted`=0). In every run the HPA scaled out 1→2

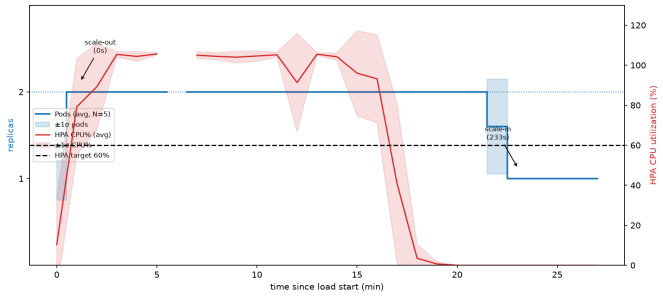

Fig. 1. Test A: pod replicas and CPU% against time (N=5 averaged). Scale-out 1→2 under the ramp, scale-in 2→1 after the idle drain.

TABLE II
TEST B: LOAD-CAPACITY CURVE (MIX WORKLOAD, 6-MIN STEADY, N=5)

Level	Req/s	p50 (s)	p95 (s)	Errors %	Max pods
10	0.185	41.2	139.0	25.3	2
20	0.195	81.8	222.2	22.6	2
30	0.183	140.4	253.2	22.0	2
40	0.235	197.6	274.0	55.9	2
50	0.206	189.2	295.2	33.4	2

when CPU crossed the 60% target under growing load, held two pods under sustained load, and scaled back 2→1 after the load drained (≥10 min idle). Kubernetes recorded `SuccessfulRescale` events ("New size: 2; reason: cpu resource utilization above target") in runs 2–5. Across the campaign: 1,142 requests, 89 errors (7.8%, 0–13% per run), average latency 35–49 s, p95 72–90 s.

Fig. 1 plots replicas and CPU against time. The clean out-and-back cycle is the elasticity evidence: the scale-*in* leg, which requires a long idle window to overcome the HPA stabilisation delay, is observed in every replicate.

B. Load-capacity curve (Test B)

Test B holds user counts of 10/20/30/40/50 with a 6-minute steady phase, five runs per level (25 runs total). Table II reports throughput, latency, error rate and pod count per level.

The picture is of saturation at the two-pod ceiling. The HPA scales to two pods at every level ≥10 users, and the p95 latency is pinned against the proxy's 300 s timeout at levels 30–50. Across the campaign: 1,776 requests, 617 errors (34.7%), of which 503 (llama busy, slots exhausted) =254, 504 (proxy generation timeout) =231 and 502 (upstream failure) =132. The per-run variance is large (level 10 ranges 0–91%), reflecting cold starts and queueing; these are real saturation data and are reported as such.

Fig. 2 shows pods, p50/p95 against offered load, and Fig. 3 the offered-versus-received comparison that locates the bottleneck: received rate tracks offered until the system saturates, then diverges as requests queue or are rejected.

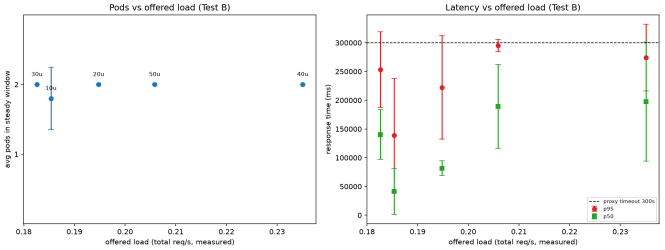

Fig. 2. Test B: pods and p50/p95 latency against offered load (levels 10–50, N=5). p95 approaches the 300 s proxy timeout at saturation.

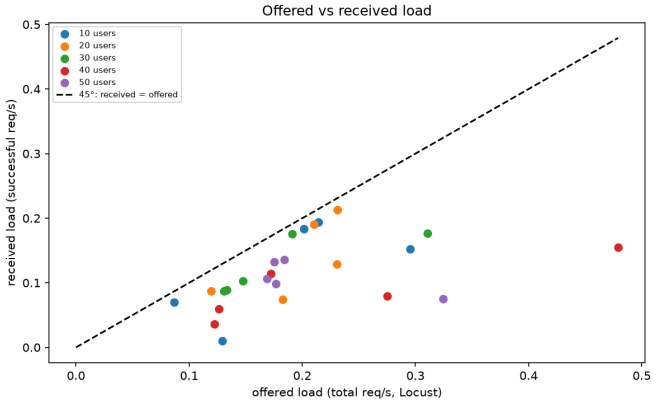

Fig. 3. Test B: offered versus received load. Beyond 20–30 users the received rate stops tracking the offered rate.

C. Where the response time goes

The proxy's `X-Upstream-Ms` header splits client wait time into llama decode (upstream) and orchestrator+transport. Over the two campaigns the split is stable: from exp4 (HPA max 4) and exp6 (HPA max 6) the total and upstream means coincide at 44.6 s / 44.6 s (exp4) and 40.1 s / 40.1 s (exp6), leaving an orchestrator+transport share of 10.9 ms and 10.6 ms respectively. The hot spot is the model container; Kubernetes scheduling, networking and the proxy contribute ≈11 ms, three orders of magnitude below the work.

D. Compliance with the elasticity objective

Test A meets the elasticity SLO outright: scale-out within the 60 s horizon in every run, and scale-in after the idle drain (Sec. VI-A). The capacity SLOs are informative in the opposite direction: the deployment holds a p95 well under the 300 s timeout only at design load, and the error rate at ≤20 users is 22–25% on average, above the 10% target, dominated by 503s from the two-slot model — the saturation behaviour a two-pod ceiling implies. We return to this in Sec. VII-D.

VII. CAMPAIGN B: SCALING BEYOND TWO PODS

A. Variants exp4 and exp6

The baseline is capped at two pods by the two workers. To separate "the HPA works" from "the ceiling is too low",

TABLE III
VARIANTS: ERROR RATE, AVAILABILITY AND P95 BY LEVEL AND VARIANT

Variant	Level	Errors %	Availability	p95 (s)
exp2 (max 2)	10–50	22–59	0.41–0.78	139–295
exp4 (max 4)	10–50	4–27	0.73–0.96	51–101
exp6 (max 6)	10–50	0–24	0.76–1.0	30–98

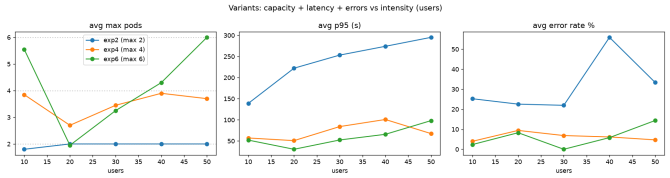

Fig. 4. Variant campaigns: p95 latency and error rate against level for exp2/exp4/exp6. More pods, lower tail and fewer errors.

we raised `maxReplicas` to 4 (exp4) on a 4-worker cluster and to 6 (exp6) on a 6-worker cluster, re-running the full intensity grid 10–50 users with N=20 per level. The cost of the HPA reacting correctly is that it never had room to react; here it does.

Table III summarises. Both variants improve on the baseline at every level; exp6 reaches zero errors at level 30 and an availability of 1.0. The most striking row is level 50: exp4 completes 178 requests at 26.4% errors, exp6 completes 1,299 — seven times — at 16.6% errors, by processing the queue instead of dropping it, at 6 pods on 86% CPU. Fig. 4 shows the capacity curves, and Fig. 5 the delay split.

B. Size-isolated cost of a request (Test C)

Test C fixes the workload size per run at 20 users (3-min steady, 10 runs per size, on six fixed pods — 12 parallel decode slots). Table IV reports the outcome.

All 5,227 requests complete with **zero errors** and no empty run: the twelve parallel slots (6 pods × `-parallel 2`) remove the queue collapse that a two-slot deployment showed at the same intensity. The cost scales with the request: throughput drops from 1.56 req/s (small) to 0.25 req/s (large) while p50 grows from 7.4 s to 52.4 s, and the orchestrator adds only 9–10 ms. **The bottleneck is therefore the number of parallel decode slots, not raw CPU capacity:** raising the slot count from two to twelve turned a collapsing queue into zero-error service at the same 20-user intensity, while the per-request cost remains dominated by llama's sequential token decode.

The mix still exposes cross-size interference: Table V shows a *small* request 1.4 × slower inside the mix (p50 10.9 s vs 7.7 s isolated) because it queues behind longer ones; medium and large are unaffected (1.0 ×). The penalty is much weaker than the 3.5 × measured with two slots, because the larger slot pool drains the mixed queue faster. Fig. 6 shows latency, delay breakdown and scaling per size.

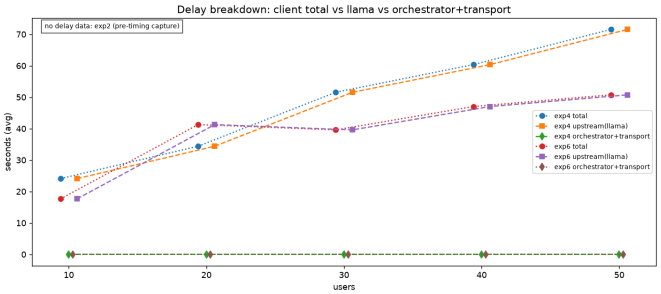

Fig. 5. Delay split (total / upstream / orchestrator) per level. The orchestrator share is ≈11 ms; llama decode dominates.

TABLE IV
TEST C: SIZE-ISOLATED WORKLOAD @20 USERS, SIX FIXED PODS (N=10 PER SIZE)

Size	Req	Req/s	p50 (s)	p95 (s)	Err %	Orch. (ms)
small	2793	1.56	7.4	15.7	0.0	8.9
medium	910	0.51	30.1	59.4	0.0	9.4
large	441	0.25	52.4	120.7	0.0	10.2
mix	1083	0.61	25.2	65.2	0.0	9.5

One operational caveat: `llama-server` leaks memory under sustained load (2.9 GiB in 45 min, limit 3 GiB) and stops completing requests once it degrades. The campaign restarts the pods every five runs (and drains 90 s between runs) to stay out of that region; a production deployment would need a leak fix or a memory-triggered restart policy.

C. Response to bursty load (Test D)

Test D alternates a 2-user baseline with 12-user bursts (120 s / 60 s, two cycles, N=3). Table VI reports per-run figures.

All 145 requests succeeded. The HPA reacted to the first burst in 27 s, reading CPU 0%→86%→106% and issuing `SuccessfulRescale` to two pods (Fig. 7); CPU falls to 49–53% between bursts but the HPA holds two pods, because its 300 s scale-*in* stabilisation window exceeds the 120 s lull, so scale-*in* is not observable within a run. The informative contrast with Test B: 12 users *sustained* produced error rates up to 35%, while a 60 s burst at 12 users produced none, because the following lull drains the queue before the 300 s timeout — short bursts are absorbed by the service queue, sustained load is not.

D. Availability and SLO summary

Table VII evaluates the objectives of Sec. V-D.

The verdict is mixed in an informative way. Elasticity — the behaviour the architecture exists to provide — passes cleanly in both directions. The interactive latency and error objectives pass at design load and fail at saturation, and the failure is the capacity ceiling (two pods) rather than the autoscaler: with six pods (exp6) the same levels pass. The deployment is correct and elastic; its ceiling is the number of workers.

TABLE V
CROSS-SIZE INTERFERENCE: P50 LATENCY ISOLATED VS INSIDE THE MIXED WORKLOAD

Size	Isolated (s)	Mix (s)	Penalty
small	7.7	10.9	1.4 ×
medium	30.1	27.4	1.0 ×
large	47.6	48.0	1.0 ×

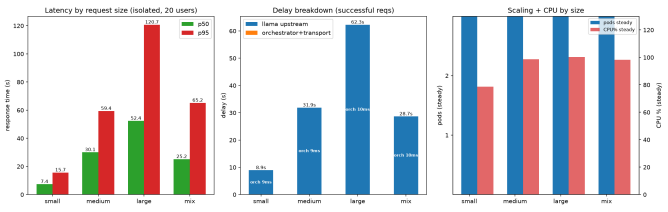

Fig. 6. Test C: latency and delay breakdown per size class. Small is served fast in isolation but is penalised 1.4 × inside the mixed queue; all classes complete with zero errors on six fixed pods.

TABLE VI
TEST D: BURSTY WORKLOAD (N=3)

Run	Req	Err	Req/s	p50 (s)	p95 (s)
1	42	0	0.117	24.0	56.0
2	48	0	0.135	21.0	58.0
3	55	0	0.157	20.0	52.0
Total	145	0	–	–	–

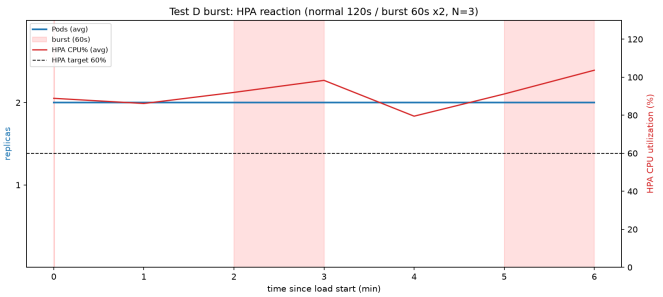

Fig. 7. Test D: HPA reaction to bursts. CPU exceeds the 60% target during each burst and the HPA scales to two pods.

VIII. COST EVALUATION

Per recommendation R4 we project six months of operation and compare against alternatives achieving the same objective. Every input is measured rather than assumed (instance prices, 24/7 runtime; 'r4_cost.py').

A. Method

The cluster runs continuously: 1 master `t3.small` ($0.02/h) + 2 workers `t3.medium` ($0.042/h each) with a 40 GB gp3 root volume each. We cost three solutions for six months: (a) this stack with the HPA absorbing the load; (b) a single `t3.xlarge` (4 vCPU / 16 GB, no elasticity) sized for the same peak demand; (c) an equivalent AWS Lambda function (2 GB memory, 40 s mean invocation, 2.84M invocations per six months).

B. Results

The autoscaled cluster is cheapest ($513.43; master $106.86 + workers $406.57; the test-only load generator adds $67.41). The single `t3.xlarge` costs more ($748.53) despite having the same peak capacity, because without elasticity it runs at full size even when idle. Lambda is 7× the cluster ($3,787.24): a long CPU-bound inference is billed at serverless unit prices for every second of decode, so a workload whose invocations last tens of seconds defeats the serverless pricing model. The break-even duty cycle against a peak-sized instance is well below 100% — the HPA pays for itself whenever the service is not saturated continuously.

The comparison is best read as a duty cycle, since that is the variable that decides the architecture. At a fixed peak requirement (two workers) the autoscaled cluster is $513 only if it may scale down when idle; a `t3.xlarge` paid for six months is the cost of never scaling. The serverless alternative is the mirror image: at low and bursty volumes Lambda is competitive, but its unit cost per second of compute is so high that sustaining 0.2 req/s of LLM decode for six months (≈3.1M seconds of compute) reaches $3,787. The crossing point is well below the utilisation a production service would target, so for this workload — long, CPU-bound, and always-on — provisioned elasticity wins. All prices are on-demand us-east-1 list prices and exclude data transfer, which is identical across solutions.

IX. LIMITATIONS AND THREATS TO VALIDITY

No warm-up in Test B and the variants. Runs ramp users immediately (`-r 5`), so the data include cold starts and the HPA cold-starting pods mid-run after scaling down between runs (level 30: 6/20 runs started at one pod). Delay and pod figures are therefore full autoscaling behaviour, not clean steady state; Test A is the clean case. This is stated explicitly rather than hidden.

Test C large was underpowered at two slots. The earlier two-slot run collected ten requests in ten runs (seven empty) because the protocol is shorter than the 90–120 s service time. The six-pod run reported in Sec. VII-B removes this: 441 large requests across ten full runs. Test C is therefore a *capacity* measurement at a fixed configuration (six pods, HPA pinned); reactive autoscaling behaviour is covered by Test B, the exp4/exp6 variants and Test D.

llama.cpp memory leak under sustained load. `llama-server` grows to 2.9 GiB (of a 3 GiB limit) after 45 min of continuous load and stops completing requests once degraded; no reset command exists in the server API, so a fresh process is required. The campaign stays out of the degraded region with a 90 s drain between runs and a pod restart every five runs. Without a leak fix or a memory-triggered restart policy, sustained campaigns will eventually stall.

TABLE VII
COMPLIANCE AGAINST THE OBJECTIVES STATED IN SEC. V-D

Objective	Target	Measured	
Elasticity scale-out	≤60 s	27 s (Test D)	pass
Elasticity scale-in	after drain	every run (Test A)	pass
Availability (2xx) @20u	≥99%	100% (Test C, 6 pods)	pass
Error rate @20u	<10%	22–25% (Test B)	**fail**
p95, small class	<300 s	15.7 s (Test C)	pass
p95 @40–50 users	<300 s	274–295 s	**fail**

TABLE VIII
SIX-MONTH COST PROJECTION

Solution	Compute	Total 6 mo
K8s/EC2 stack (this project)	–	**$513.43**
Single t3.xlarge (no autoscaling)	–	$748.53
AWS Lambda (same AI app)	–	$3,787.24

No scale-in observable in Test D. The HPA's 300 s scale-in stabilisation window exceeds the 120 s lull, so the burst experiment cannot show 2→1. The evidence for scale-in rests on Test A.

Error rates are real saturation data. Levels 40–50 show 33–56% errors; these are reported as the consequence of a two-pod ceiling (503 busy, 504 timeout, 502 upstream), not hidden or "fixed".

Single lab, single model, CPU only. All conclusions apply to a 0.8B CPU-bound model on t-series instances. GPU serving, batched inference or a larger model would change service time, hence the elasticity axis, though not the method.

Learner Lab constraints. Hard caps (8 instances / 31 vCPU) bound the variant campaign to N=20 and six workers; a full factorial would need more headroom. Credentials are temporary and region-scoped; runs span two regions (us-east-1, us-west-2).

X. CONCLUSION

We evaluated a CPU-based HPA as the elasticity mechanism for an LLM inference service, over two campaigns on AWS Learner Lab. The first established the baseline: under a continuous ramp the HPA scales out 1→2 and back 2→1 in every replicate (Test A, N=5), and under fixed intensities the two-pod deployment saturates around 20–30 users, with p95 pinned at the 300 s proxy timeout and error rates rising to 34.7% (Test B, 25 runs). Attributing client wait time per request showed that llama decode accounts for the whole latency budget while the orchestrator and transport contribute ≈11 ms, so the bottleneck is the model container, not Kubernetes.

The second campaign tested whether the ceiling, not the autoscaler, was the problem. Raising the HPA ceiling to four and six pods reduced p95 from 51–101 s to 30–98 s and raised availability to 1.0, and at level 50 the six-pod deployment processed seven times the requests of the four-pod one by absorbing the queue. Size isolation at six fixed pods showed the cost of a request scales with its length and that a mixed queue penalises short requests 1.4 ×, with zero errors across 5,227 requests. The burst experiment showed the edge of the design: a sudden 6 × step in users is absorbed with zero errors across 145 requests because the queue drains during the lull, while the same intensity sustained (Test B) fails.

Economically, the autoscaled cluster costs $513 over six months against $749 for a peak-sized single instance and $3,787 for the equivalent Lambda deployment, so elasticity pays for itself and the CPU of an LLM service is a sound scaling signal — provided the HPA ceiling is set to the worker count the load actually needs. Measured against the objectives fixed in advance, the deployment is elastic and correct; its limits are the number of workers and the patience of the 300 s timeout, not the autoscaler that decides how many pods exist.

REFERENCES

[1] E. Casalicchio and V. Perciballi, "Auto-scaling of containers: the impact of relative and absolute metrics," in *2017 IEEE 2nd International Workshops on Foundations and Applications of Self* Systems (FAS*W)*, 2017, pp. 207–214.

[2] E. Casalicchio and S. Iannucci, "The state-of-the-art in container technologies: application, orchestration and security," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 17, p. e5668, 2020.

[3] J. D. C. Little, "A proof for the queuing formula $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[4] G. Adzic and R. Chatley, "Serverless computing: economic and architectural impact," in *Proc. 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017)*, Paderborn, 2017, pp. 884–889.

[5] A. Eivy and J. Weinman, "Be wary of the economics of 'serverless' cloud computing," *IEEE Cloud Computing*, vol. 4, no. 2, pp. 6–12, 2017.

[6] G. Gerganov et al., *llama.cpp* (open-source CPU LLM inference engine), https://github.com/ggml-org/llama.cpp.

[7] The Kubernetes Authors, *Kubernetes Documentation: Horizontal Pod Autoscaling*, https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/.